DS605: Fundamentals of Machine Learning

Lab Assignment - 3

Scikit-learn: Data Preprocessing and Model Performance Evaluation

---

Dataset: Kaggle Hotel Booking Demand (hotel_bookings.csv)
Submission: Public GitHub repository link only.

Objective

Build and compare Scikit-learn preprocessing pipelines and evaluate two classification models.

Part A - Data Loading and Preprocessing with Scikit-learn

Task 1 - Load and Understand the Dataset

- Load hotel_bookings.csv. Use head(), shape, info(), describe(), and dtypes.
- Check the class distribution of is_canceled and use it as the target y.
- Create X from the remaining usable features. Identify numerical and categorical columns.

Task 2 - Missing Values, Leakage, and Outliers

- Find the missing-value count and percentage for every column.
- Identify columns with very high missingness. Drop them only if justified and state the reason.
- Remove columns that directly reveal the final booking outcome, such as reservation_status and reservation_status_date.
- Check selected numerical features for outliers using boxplots and/or the IQR method.
- Remove only clear/extreme outliers and report how many rows were removed.

Task 3 - Create Two Preprocessing Pipelines

- Split the data using train_test_split(test_size=0.2, stratify=y, random_state=42). Use the same split for all experiments.
- For numerical columns, use KNNImputer(n_neighbors=5) for missing values.
- For categorical columns, use SimpleImputer(strategy="most_frequent") and OneHotEncoder(handle_unknown="ignore").
- Pipeline A: KNNImputer + StandardScaler for numerical features.
- Pipeline B: KNNImputer + MinMaxScaler for numerical features.
- Use ColumnTransformer and Pipeline so that preprocessing is fitted only on the training data.

Part B - Model Training and Performance Evaluation

Task 4 - Train Two Classification Models

- Train LogisticRegression(max_iter=1000) using Pipeline A and Pipeline B.
- Train DecisionTreeClassifier(random_state=42) using Pipeline A and Pipeline B.
- Keep the model settings and train-test split unchanged across the comparison.
- You should obtain four trained model-pipeline combinations in total.

Task 5 - Evaluate and Compare the Four Results

- For each experiment, calculate training accuracy, testing accuracy, precision, recall, and F1-score.
- Create a final comparison table containing all four model-pipeline combinations.
- Plot a confusion matrix for the best Logistic Regression result and the best Decision Tree result.
- Use the train-test performance difference to comment briefly on possible overfitting.

Task 6 - Final Observations

- Which preprocessing-model combination gives the best overall result?
- Does StandardScaler or MinMaxScaler affect Logistic Regression more?
- Does scaling make a major difference for the Decision Tree? Comment briefly.
- Write 4-5 short observations supported by the performance table and confusion matrices.

Deliverables (GitHub Only)

- README with assignment title, name, ID, dataset link, preprocessing choices, and final observations.
- Complete runnable Jupyter Notebook covering data cleaning, both pipelines, both models, and evaluation.
- Final comparison table, required confusion-matrix figures, and the cleaned base dataset used for modeling.
- Submit only the public GitHub repository link.